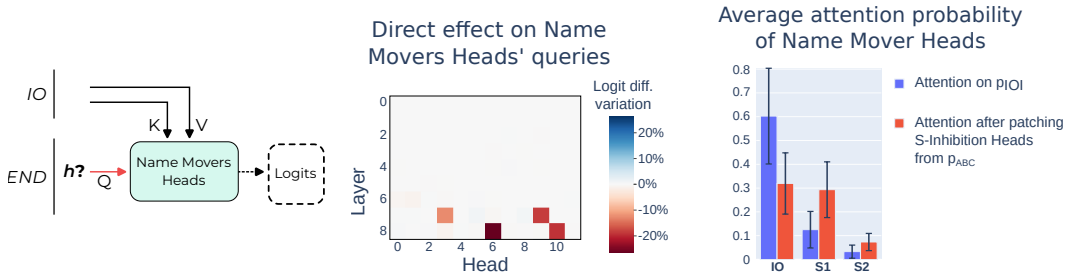
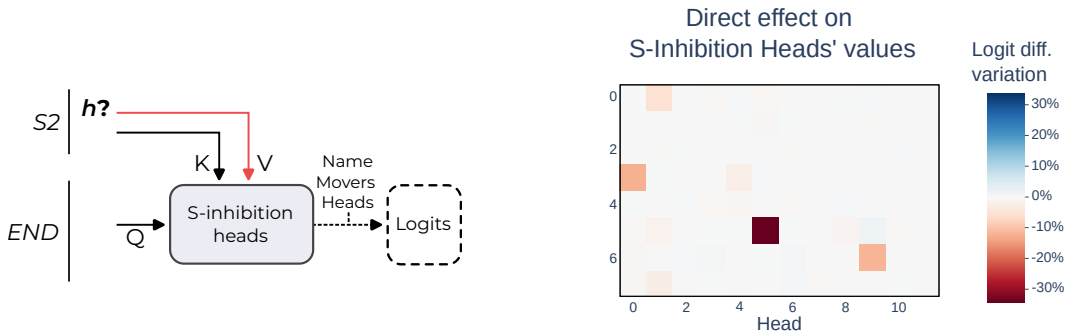


(a) (b) (c)

Figure 4: (a) Diagram of the direct effect experiments on Name Mover Heads' queries. We patch in the red path from the p_{ABC} distribution. Then, we measure the indirect effect of h on the logits (dotted line). All attention heads are recomputed on this path. (b) Result of the path patching experiments. The four heads causing a decrease in logit difference are S-Inhibition Heads. (c) Attention of Name Mover Heads on p_{IOI} before and after path patching of S-Inhibition Heads $\rightarrow$ Name Mover Heads' queries for all four S-Inhibition Heads at once (black bars show the standard deviation). S-Inhibition Heads are responsible for Name Mover Heads' selective attention on IO.



(a) (b)

Figure 5: (a) Diagram of the direct effect experiment on S-Inhibition Heads' values. On the indirect path from h to the logits (dotted line), all elements are recomputed, Name Movers Heads are mediating the effect on logits. (b) Result of the path patching experiments for the S-Inhibition Heads' values.

the effect on logit difference in Figure 4b. Heads 7.3, 7.9, 8.6, 8.10 directly influence the Name Mover Heads' queries, causing a significant drop in logit difference through this path.

S-Inhibition Heads. To better understand the influence of these heads on the Name Movers Heads, we visualized Name Movers Heads' attention before and after applying path patching of all four previously identified heads at once (Figure 4c). Patching these heads leads to a greater probability on S1 (and thus less on IO). We therefore call these heads *S-Inhibition Heads*, since their effect is to decrease attention to S1.

What are S-Inhibition heads writing? We observed that S-Inhibition heads preferentially attend to the S2 token (the average attention from END to S2 was 0.51 over the four heads). We conjectured that they move information about S that causes Name Movers to attend less to S1. We discovered two signals through which S-Inhibition Heads do this. The first, the *token signal*, contains the value of the token S and causes Name Mover Heads to avoid occurrences of that token. The second, the *position signal*, contains information about the position of the S1 token, causing Name Mover Heads to avoid the S1 position no matter the value of the token at this position.

To disentangle the two effects, we designed a series of counterfactual experiments where only some signals are present, and the others are inverted with respect to the original dataset (see Appendix A). Both effects matter, but position signals have a greater effect than token signals. These two signals suffice to account for the full average difference in Name Mover Heads' attention to IO over S1.

S-Inhibition heads output information about the position of S1 but don't attend to it directly. They thus must rely on the output of other heads through value composition, as we will see next.

3.3 Which heads affect S-Inhibition values?

Tracing back the information flow. We next trace the information flow back through the queries, keys, and values of the S-Inhibition Heads, using path patching as before. We found the values were the most important and so focus on them here, with keys discussed in Appendix C (queries had no significant direct effect on the S-Inhibition Heads).

In more detail, for each head h at the S2 position we patched in the path $h \rightarrow$ S-Inhibition Heads's values (Figure 5a) and reported results in Figure 5b.

We identified 4 heads with a significant negative effect on logit difference. By looking at their attention pattern, we separated them into two groups. The heads from the first group attend from S2 to S1, while those from the second group attend from S2 to the token after S1 (S1+1 in short).

Duplicate Token Heads. Two of the heads attend from S2 to S1. We hypothesize that these heads attend to the previous occurrence of a duplicate token and copy the position of this previous occurrence to the current position. We therefore call them *Duplicate Token Heads*.

To partially validate this hypothesis, we investigated these heads' attention patterns on sequences of random tokens (with no semantic meaning). We found that the 2 Duplicate Token Heads pay strong attention to a previous occurrence of the current token when it exists (see Appendix I).

Induction Heads and Previous Token Heads. The other group of two heads attends from S2 to S1+1. This matches the attention pattern of *Induction Heads* (Elhage et al., 2021), which recognize the general pattern `[A] [B] ... [A]` and contribute to predicting `[B]` as the next token. For this, they act in pairs with Previous Token Heads. The Previous Token Heads write information about `[A]` into the residual stream at `[B]`. Induction Heads can then match the next occurrence of `[A]` to that position (and subsequently copy `[B]` to the output).

To test the hypothesis that this group of heads consists of Induction Heads, we sought out the Previous Token Heads that they rely on through key composition. To this end, we apply path patching to find heads that influence logit difference through the S1+1 keys of the Induction Heads (see Appendix D). We identified two heads with significant effect sizes. To test the induction mechanism, we followed the method introduced in Olsson et al. (2022) by checking for prefix-matching (attending to `[B]` on pattern like `[A] [B] ... [A]`) and copying (increasing the logit of the `[B]` token) of Induction Heads on repeated random sequences of tokens. We also analyzed the attention patterns of Previous Token Heads on the same dataset. As reported in Appendix H, Previous Token Heads attend primarily to the previous token and 2 Induction Heads demonstrate both prefix-matching and copying.

Locating the position signal. Despite their differences, the attention patterns of both groups of heads are determined by the position of S1, suggesting that their output could also contain this information. We thus hypothesized that Duplicate Token Heads and Induction Heads are the origin of the position signal described in Section 3.2, which is then copied by S-Inhibition Heads.

To test this hypothesis, we applied the same technique from the previous section to isolate token and position signals. We patched the output of the Induction and Duplicate Token Heads from sentences with altered position signals. We observed a drop in logit difference of almost the same size as patching the S-Inhibition Heads (at least 88%). However, patching their output from a dataset with a different S1 and S2 token but at the same position doesn't change the logit difference (less than 8% of the drop induced by patching S-Inhibition Heads).

Missing validations and caveats. To fully validate the claimed function of the Duplicate Token and Induction Heads, we would want to perform additional checks that we omitted due to time constraints. First, for Duplicate Token Heads, we would want to study the interaction of the QK matrix with the token embeddings to test if it acts as a collision detector. Moreover, we would investigate the OV matrix to ensure it is copying information from positional embeddings. For Induction Heads, we would perform the more thorough parameter analysis in Elhage et al. (2021) to validate the copying and key composition properties at the parameter level.

Moreover, in the context we study, Induction Heads perform a different function compared to what was described in their original discovery. In our circuit, Induction Heads' outputs are used as positional signals for S-Inhibition values. We did not investigate how this mechanism was implemented. More work is needed to precisely characterize these newly discovered heads and understand how they differ from the original description of Induction Heads.

3.4 Did we miss anything? The Story of the Backup Name Movers Heads

Each type of head in our circuit has many copies, suggesting that the model implements redundant behavior[2]. To make sure that we didn't miss any copies, we knocked out all the Name Mover Heads at once. To our surprise, the circuit still worked (only 5% drop in logit difference). After the knockout, we found the new heads directly affecting the logits using the same path patching experiment from Section 3.1. These new heads compensate for the loss of Name Movers Heads by replacing their role.

We selected the eight heads with the highest direct influence on logit difference after knockout and call them *Backup Name Mover Heads*. We investigated their behavior before the knockout. We observe diverse behavior: 4 heads show a close resemblance to Name Mover Heads, 2 heads equally attend to IO and S and copy them, 1 head pays more attention to S1 and copies it, and 1 head seems to track and copy subjects of clauses, copying S2 in this case. See Appendix F for further details on these heads.

We hypothesize that this compensation phenomenon is caused by the use of dropout during training. Thus, the model was optimized for robustness to dysfunctional parts. More work is needed to determine the origin of this phenomenon and if such compensation effects can be observed beyond our special case.

4 Experimental validation

In the previous section, we provided an end-to-end explanation of how the model produces a high logit difference between IO and S on the IOI task. Yet Section 3.4 shows that our experimental methods do not fully account for all name-moving in the model: some new heads take on this role after the main Name Mover Heads are knocked out. We therefore seek more systematic validations to check that our account in Section 2.1 includes all components used by the model to perform IOI.

To formalize this, we first introduce notation to measure the performance of a circuit C on the IOI task. Suppose $X \sim \mathsf{p}_{\text{IOI}}$, and $f(C(X); X)$ is the logit difference between the IO and S tokens when the circuit C is run on the input X (recall that $C(X)$ is defined by knocking out all nodes outside C, as described in Section 2.1). We define the average logit difference $F(C) \stackrel{\text{def}}{=} \mathbb{E}_{X \sim \mathsf{p}_{\text{IOI}}} [f(C(X); X)]$ as a measure of how well C performs the IOI task.

Using F, we introduce three criteria for validating a circuit C. The first one, **faithfulness**, asks that C has similar performance to the full model M, as measured by $|F(M) - F(C)|$. For the circuit C from Figure 2, we find that $|F(M) - F(C)| = 0.46$, or only 13% of $F(M) = 3.56$. In other words, C achieves 87% of the performance of M.

However, as the Backup Name Mover Heads illustrate, faithfulness alone is not enough to ensure a complete account. In Section 4.1 we introduce a toy example that illustrates this issue in more detail and use this to define two other criteria—completeness and minimality—for checking our circuit.

4.1 Completeness

As a running example, suppose a model M uses two identical and disjoint serial circuits C_1 and C_2. The two sub-circuits are run in parallel before applying an OR operation to their results. Identifying only one of the circuits is enough to achieve faithfulness, but we want explanations that include both C_1 and C_2, since these are both used in the model's computation.

[2]It is unlikely that SGD would select for perfectly redundant modules, but redundancy is more likely when analyzing a sub-distribution of all of a model's training data or a subtask of its behavior.